

Arsitektur Aplikasi CPR Detector (Offline + React Native)

Ringkasan Arsitektur

Aplikasi ini adalah 100% React Native dengan TFLite offline!

```
mobile-cpr-detector/ (1 REPOSITORY)
├─ Frontend: React Native (JavaScript)
├─ ML Engine: TensorFlow Lite (Native)
└─ Bridge: react-native-fast-tflite (Kotlin/Java)
```

Pertanyaan yang Sering Muncul

? "Apakah aku harus menulis Kotlin?"

TIDAK! Kamu hanya menulis JavaScript React Native. Kotlin sudah disediakan oleh library.

? "Apakah bisa offline?"

YA! Model .tflite ada di dalam aplikasi, tidak perlu internet.

? "Apakah ini 1 repository?"

YA! Semua ada di folder `mobile-cpr-detector/`.

? "Apakah frontendnya React Native?"

YA! Semua UI adalah React Native (CameraScreen, CPROverlay, dll).

? "Bagaimana .tflite bisa jalan di React Native?"

Lewat native bridge! Library `react-native-fast-tflite` menghubungkan JavaScript dengan TFLite native.

Cara Kerja Layer by Layer

Layer 1: React Native (Yang Kamu Tulis - JavaScript)

File yang kamu kerjakan:

```
components/
├─ CameraScreen.js      ← Ambil gambar dari kamera
├─ CPROverlay.js        ← Tampilkan overlay CPR point
```

```
services/  
└─ MoveNetTFLiteDetector.js ← Panggil model TFLite
```

Contoh kode yang kamu tulis (JavaScript):

```
import { loadTensorflowModel } from 'react-native-fast-tflite';  
  
// Load model TFLite  
const model = await loadTensorflowModel('/path/to/4.tflite');  
  
// Jalankan deteksi pose  
const keypoints = await model.run([imageData]);  
  
// Hasilnya langsung kamu pakai di React Native  
console.log('Keypoints:', keypoints);
```

Kamu HANYA menulis JavaScript seperti ini!

Layer 2: Native Bridge (Sudah Ada di Library)

File Kotlin/Java (TIDAK PERLU KAMU EDIT):

```
node_modules/react-native-fast-tflite/android/  
└─ src/main/java/com/tflite/  
    └─ TfliteModule.java      ← Bridge JavaScript → Native  
    └─ TflitePackage.java    ← Register module
```

Apa yang terjadi di sini:

1. Kamu panggil `loadTensorflowModel()` di JavaScript
2. Bridge menerjemahkan ke kode native
3. Native code memanggil TensorFlow Lite C++ library
4. Hasil dikembalikan ke JavaScript

Kamu TIDAK PERLU menyentuh file ini!

Layer 3: TensorFlow Lite (C++ Library)

File binary:

```
node_modules/react-native-fast-tflite/android/  
└─ libs/  
    └─ libtensorflowlite.so ← TFLite engine (compiled)
```

Apa yang terjadi:

- Load file **4.tflite** dari assets
- Parse model architecture
- Jalankan inferensi pose detection
- Return keypoints ke JavaScript

Ini sudah compiled binary, tidak perlu kamu sentuh!

Struktur File di Repository

```
mobile-cpr-detector/ ← 1 REPOSITORY SAJA
|
├─ App.js                ← Entry point React Native
├─ package.json          ← Dependencies (termasuk react-native-
fast-tflite)
|
├─ assets/
│   └─ models/
│       └─ 4.tflite      ← Model TFLite OFFLINE (8.3 MB)
|
├─ components/           ← UI React Native (JavaScript)
│   ├── CameraScreen.js  ← Kamera + deteksi
│   └─ CPROverlay.js     ← Overlay visual
|
├─ services/             ← Business logic (JavaScript)
│   └─ MoveNetTFLiteDetector.js ← Load & run TFLite model
|
├─ utils/
│   └─ cprCalculation.js ← Hitung CPR point
|
├─ android/              ← Native Android (AUTO-GENERATED)
│   ├── app/
│   │   └─ src/main/java/com/cprdetector/app/
│   │       ├── MainActivity.kt    ← Entry point Android
│   │       └─ MainApplication.kt  ← Setup React Native
│   └─ build.gradle             ← Build config
|
└─ node_modules/
    └─ react-native-fast-tflite/ ← Library bridge (TIDAK PERLU
EDIT)
        └─ android/
            └─ src/main/java/com/tflite/
                └─ TfliteModule.java ← Native bridge code
```

PROF

Flow Eksekusi (Saat Deteksi Pose)

1 User Buka Aplikasi

React Native App Start
↓
MainActivity.kt (Kotlin) - AUTO
↓
App.js (JavaScript) - KAMU TULIS
↓
CameraScreen.js (JavaScript) - KAMU TULIS

2 Load Model TFLite

```
// MoveNetTFLiteDetector.js (KAMU TULIS)
async initialize() {
  // 1. Copy model dari assets ke document directory
  const modelAsset =
    Asset.fromModule(require('../assets/models/4.tflite'));
  await modelAsset.downloadAsync();

  const modelPath = `${FileSystem.documentDirectory}4.tflite`;
  await FileSystem.copyAsync({
    from: modelAsset.localUri,
    to: modelPath
  });

  // 2. Load model via native bridge
  this.model = await loadTensorflowModel(modelPath);
  //                               ↑
  //                               Ini memanggil TfliteModule.java (Kotlin)
  //                               Yang memanggil TFLite C++ library
  //                               Semua otomatis!

  this.isReady = true;
  console.log('✔ Model TFLite loaded!');
}
```

PROF

Alur di balik layar:

JavaScript: loadTensorflowModel(path)
↓
Bridge: TfliteModule.fetchByteDataFromUrl() (Java/Kotlin)
↓
Native: TensorFlow Lite C++ - load model
↓
Bridge: Return model object ke JavaScript
↓
JavaScript: model.run() siap digunakan

3 Deteksi Pose dari Kamera

```
// CameraScreen.js (KAMU TULIS)
const detectPose = async () => {
  // 1. Ambil frame dari kamera (nanti kita implement)
  const imageData = await captureFrame();

  // 2. Jalankan deteksi via native TFLite
  const keypoints = await detector.detectPose(imageData);
  //                               ↑
  //                               Ini memanggil model.run() di
  MoveNetTFLiteDetector
  //                               Yang memanggil native TFLite inference

  // 3. Hasil langsung kamu pakai di React Native
  setCprPoint(keypoints.cprPoint);
  setKeypoints(keypoints.keypoints);

  // 4. UI otomatis update
};
```

Alur inferensi:

```
JavaScript: model.run([imageData])
  ↓
Bridge: TfliteModule.run() (Java/Kotlin)
  ↓
Native: TFLite inference (C++)
  - Preprocess image: resize 192x192
  - Run neural network
  - Extract 17 keypoints
  ↓
Bridge: Convert keypoints ke JavaScript array
  ↓
JavaScript: return [[y, x, score], ...]
  ↓
React Native: Render overlay di UI
```

PROF

Apa yang Kamu Tulis vs Apa yang Auto

 Yang KAMU TULIS (JavaScript React Native)

File: `services/MoveNetTFLiteDetector.js`

```
import { loadTensorflowModel } from 'react-native-fast-tflite';
```

```

class MoveNetDetector {
  async initialize() {
    // Load model
    this.model = await loadTensorflowModel(modelPath);
  }

  async detectPose(imageData) {
    // Run inference
    const outputs = await this.model.run([imageData]);
    return this.parseModelOutput(outputs[0]);
  }

  parseModelOutput(rawOutput) {
    // Parse keypoints [1,1,17,3] → [[y,x,score], ...]
    // Hitung CPR point
    // Return ke UI
  }
}

```

File: components/CameraScreen.js

```

import MoveNetDetector from '../services/MoveNetTFLiteDetector';

const detector = new MoveNetDetector();
await detector.initialize(); // Load model sekali

const keypoints = await detector.detectPose(frameData);
setCprPoint(keypoints.cprPoint); // Update UI

```

Semua ini JavaScript React Native!

—
PROF

 Yang AUTO (Kotlin/Java dari Library)

File: node_modules/react-native-fast-tflite/android/.../TfliteModule.java

```

// KAMU TIDAK PERLU EDIT INI!
// Library sudah menyediakan

@ReactMethod
public void loadTensorflowModel(String modelPath, Promise promise) {
  try {
    // Load .tflite file
    ByteBuffer modelBuffer = loadModelFile(modelPath);

    // Initialize TFLite interpreter
    Interpreter interpreter = new Interpreter(modelBuffer);

```

```

    // Return ke JavaScript
    promise.resolve(interpreter);
} catch (Exception e) {
    promise.reject(e);
}
}

@ReactMethod
public void run(byte[] imageData, Promise promise) {
    // Run TFLite inference
    float[][] output = new float[1][51]; // [1,1,17,3] flattened
    interpreter.run(imageData, output);

    // Return ke JavaScript
    promise.resolve(output);
}

```

Kode Kotlin/Java ini sudah ada di library!
Kamu TIDAK perlu menulis ini!

🔌 Kenapa Butuh Native Bridge?

Masalah:

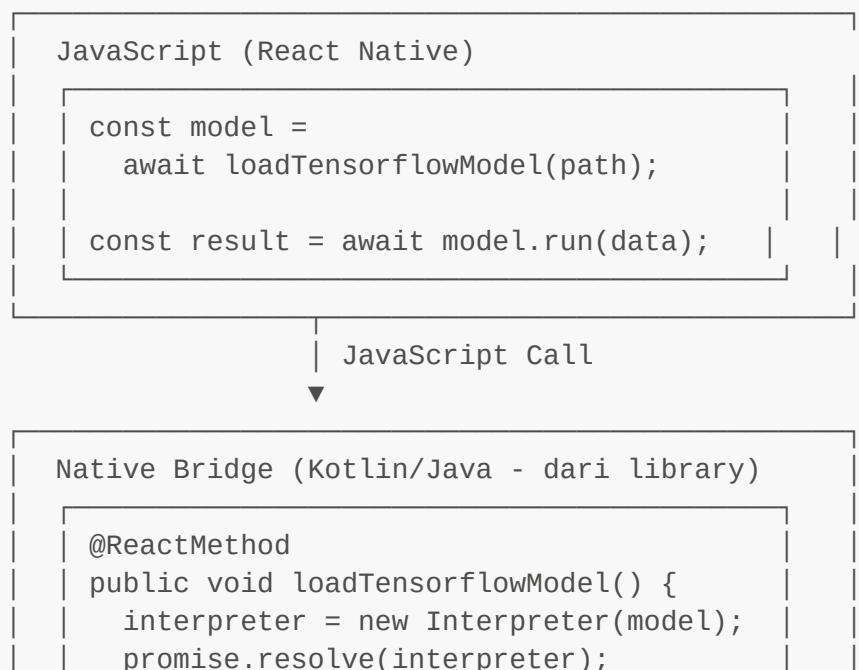
TensorFlow Lite adalah **C++ library** yang hanya bisa jalan di native code.

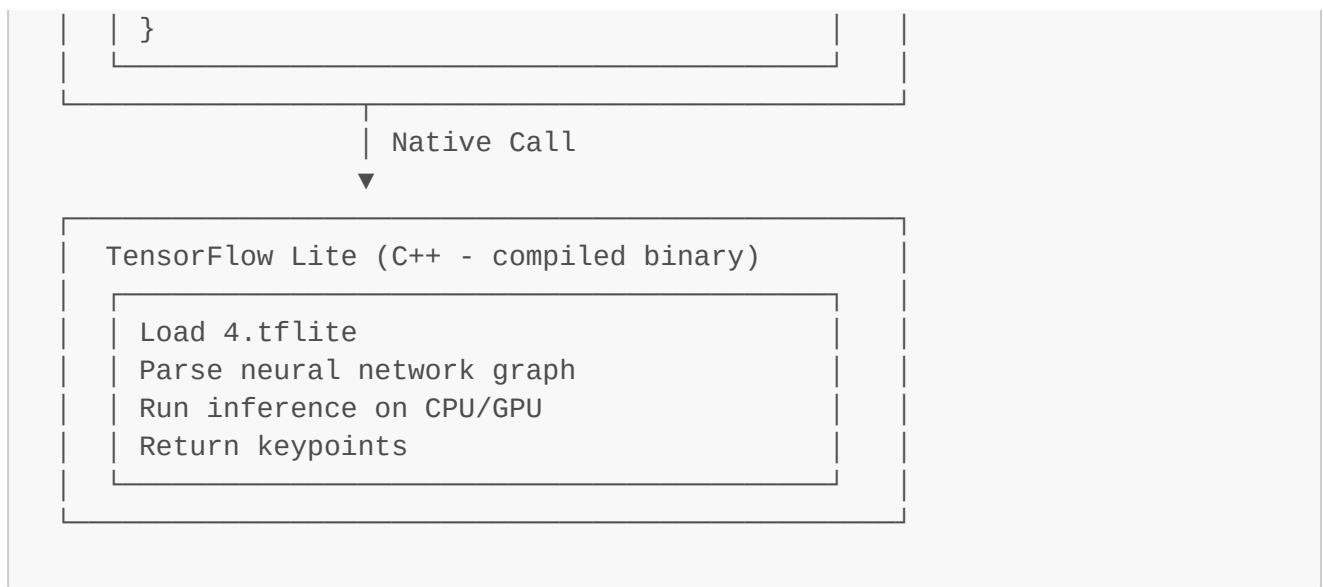
React Native adalah **JavaScript** yang jalan di JavaScript engine.

Solusi:

Library **react-native-fast-tflite** menyediakan **bridge** antara keduanya.

PROF





Kamu hanya perlu tahu cara menggunakan bridge di JavaScript!

📱 Apakah Aplikasi Tetap Offline?

✓ YA! 100% Offline

File yang di-bundle dalam APK:

```
app.apk (compressed)
├─ JavaScript code (App.js, CameraScreen.js, dll)
├─ React Native runtime
├─ assets/models/4.tflite ← Model ML (8.3 MB)
├─ libtensorflowlite.so ← TFLite engine (native)
└─ TfliteModule.class ← Bridge code (compiled)
```

Saat aplikasi jalan:

1. User buka app → Tidak perlu internet
2. Load model dari assets → Tidak perlu internet
3. Ambil frame dari kamera → Tidak perlu internet
4. Jalankan inferensi TFLite → Tidak perlu internet
5. Tampilkan hasil di UI → Tidak perlu internet

Semua proses lokal di device!

🔧 Yang Sudah Kita Implementasikan

✓ Sudah Selesai:

1. **React Native setup** - App.js, CameraScreen, CPROverlay
2. **TFLite integration** - MoveNetTFLiteDetector.js

3. **Native bridge** - react-native-fast-tflite installed
4. **Model preparation** - 4.tflite di assets/models/
5. **Build configuration** - EAS Build untuk compile native code

Sedang Proses:

- **EAS Build** - Compile Kotlin/Java bridge code → APK
 - Status: Files uploading (1.1 MB / 9.8 MB)
 - Estimasi: ~10-15 menit

Next Steps:

1. Download APK dari EAS Build
2. Install di HP Android
3. Test model loading
4. **Implement camera frame capture** (JavaScript, bukan Kotlin!)
5. Test real-time pose detection

Kode yang Perlu Kamu Tulis Selanjutnya

Task Berikutnya: Camera Frame Capture

File: `components/CameraScreen.js` (React Native JavaScript)

```
import * as ImageManipulator from 'expo-image-manipulator';

const captureFrame = async () => {
  // 1. Ambil snapshot dari kamera
  const photo = await cameraRef.current.takePictureAsync({
    quality: 0.5,
    base64: false,
  });

  // 2. Resize ke 192x192 (input model)
  const resized = await ImageManipulator.manipulateAsync(
    photo.uri,
    [{ resize: { width: 192, height: 192 } }],
    { format: 'jpeg' }
  );

  // 3. Convert ke format yang TFLite terima
  const imageData = await convertToTensorInput(resized.uri);

  return imageData;
};

// Panggil deteksi setiap frame
const detectLoop = async () => {
  const frame = await captureFrame();
```

```
const keypoints = await detector.detectPose(frame);

// Update UI
setCprPoint(keypoints.cprPoint);
};
```

Ini semua JavaScript React Native!

Tidak ada Kotlin yang perlu kamu tulis!

Kesimpulan

Apa yang Benar:

- ✓ Frontend: React Native (JavaScript)
- ✓ Offline: 100% offline
- ✓ 1 Repository: mobile-cpr-detector/
- ✓ TFLite: Bisa digunakan via native bridge
- ✓ Kamu menulis: JavaScript saja

Apa yang SALAH:

- × "Harus menulis Kotlin sendiri" → SALAH! Library sudah menyediakan
- × "Tidak bisa pakai .tflite di React Native" → SALAH! Bisa via bridge
- × "Butuh backend terpisah" → SALAH! Semua di 1 app
- × "Butuh internet" → SALAH! 100% offline

Solusi yang Sudah Kita Implementasikan:

```
React Native (JavaScript - kamu tulis)
    ↓
react-native-fast-tflite (bridge - library sediakan)
    ↓
TensorFlow Lite (native - auto compile)
    ↓
4.tflite (model - offline di assets)
```

SEMUA SUDAH BENAR!

Tinggal tunggu EAS Build selesai, lalu implement camera frame capture dalam JavaScript!

Pertanyaan Lanjutan?

Jika masih ada yang membingungkan, tanyakan:

1. Bagian mana yang masih bingung?
2. Ingin lihat kode spesifik yang mana?

3. Ingin penjelasan tentang layer tertentu?

Kamu TIDAK perlu belajar Kotlin untuk project ini!
Semua yang kamu tulis adalah React Native JavaScript!